

CSED 312 Project 1 Final Report

김지현(20220302) 전현빈(20220259)

1. Alarm Clock

Implementation & Improvement from the previous design

Goal:

기존의 Pintos는 busy waiting 방식으로 구현되어 있다. Busy waiting 방식에서는 `timer_sleep` 함수 내에서 while 루프를 반복하며 현재 시간을 계속 확인하고, 충분한 시간이 경과하면 `thread_yield` 함수를 호출하여 CPU를 다른 thread에게 넘겨준다. 하지만 이러한 방식은 while 루프의 조건을 지속적으로 확인해야 하므로 비효율적이다.

이번 과제에서는 busy waiting의 비효율성을 개선하기 위해, 과제 설명에 나온 `thread_block`과 `thread_unblock` 함수를 사용하여 코드를 보다 효율적으로 수정하였다.

`thread_block` 함수는 현재 thread의 상태를 `THREAD_BLOCKED`로 변경한 후, CPU 실행 priority를 다시 스케줄링한다. 이는 thread가 불필요하게 CPU를 차지하는 것을 방지하는 효과가 있다. `thread_unblock` 함수는 차단된 thread의 상태를 `THREAD_READY`로 변경하고, 해당 thread를 준비 리스트에 추가하여 다시 실행될 수 있도록 한다.

이러한 방식으로 코드를 개선함으로써, busy waiting 없이 thread가 효율적으로 차단되었다가 재개될 수 있게 된다.

```
void
thread_block (void)
{
    ASSERT (!intr_context ());
    ASSERT (intr_get_level () == INTR_OFF);

    thread_current ()->status = THREAD_BLOCKED;
    schedule ();
}

void
thread_unblock (struct thread *t)
{
    enum intr_level old_level;

    ASSERT (is_thread (t));

    old_level = intr_disable ();
    ASSERT (t->status == THREAD_BLOCKED);
    list_push_back (&ready_list, &t->elem);
    t->status = THREAD_READY;
    intr_set_level (old_level);
}
```

위에서 설명한 함수들을 사용하여 각 thread별로 sleep할 시간을 저장하고, 해당 시간이 지나면 다시 thread를 awake하는 방식으로 구현하였다. 이를 위해 각 thread가 얼마나 오랫동안 sleep 상태로 있을지를 저장할 정수형 변수를 선언해야 한다. 따라서, 아래 코드와 같이 `thread` 구조체에 `wakeup_tick`이라는 정수형 변수를 선언하였다.

```
struct thread
{
    /*******/
    int64_t wakeup_tick;
    /*******/

    /* Owned by thread.c. */
    tid_t tid;                /* Thread identifier. */
    enum thread_status status; /* Thread state. */
    .
    .
    .
};
```

그리고 `THREAD_READY` 상태에 있는 threads를 관리하는 `ready_list`가 존재하는 것처럼, `THREAD_BLOCKED` 상태의 threads를 리스트 형태로 저장하는 `blocked_list`를 정의하여 `thread_init` 함수에서 다른 리스트들과 함께 초기화해준다.

```
static struct list blocked_list;

void
thread_init (void)
{
    ASSERT (intr_get_level () == INTR_OFF);
    lock_init (&tid_lock);
    list_init (&ready_list);
    list_init (&all_list);
    /*******/
    list_init (&blocked_list);
    /*******/
    .
    .
    .
}
```

이후 새로운 두 함수를 정의하였다. 첫번째로, `thread_sleep` 함수에서는 현재 thread `t`가 idle thread인지 확인한 후, 현재 thread의 `wakeup_tick` 변수에 몇 틱(tick) 후에 깨어날지를 설정한다. 이후 현재 thread를 `blocked_list`에 추가하고, `thread_block` 함수를 호출하여 현재 thread의 상태를 `THREAD_BLOCKED`로 전환시킨다.

이렇게 thread의 상태를 변경하고 `blocked_list`에 추가하는 과정은 여러 thread가 동시에 접근할 수 있는 shared resource에 접근하는 과정이기 때문에 atomic 하게 이루어져야 한다. 만약 다른 thread가 CPU를 점유하여 이 shared resource에 접근하면 race condition이 발생할 수 있다. 따라서 이 과정 전에 `intr_disable` 함수를 사용하여 인터럽트를 비활성화하고, 모든 과정이 끝난 후에는 `intr_set_level` 함수를 이용해 인터럽트를 다시 활성화한다.

```
void
thread_sleep(int64_t ticks)
{
    struct thread *t;
    enum intr_level old_level;

    old_level = intr_disable();
    t = thread_current();

    ASSERT(t != idle_thread);

    t->wakeup_tick = ticks;
    list_push_back(&blocked_list, &t->elem);
    thread_block();

    intr_set_level(old_level);
}
```

반면, `thread_awake` 함수는 차단된 thread가 깨어날 시점에 thread의 상태를 `THREAD_READY`로 변경하고, 실행 가능 상태로 `ready_list`에 추가하는 과정을 포함하고 있다. 먼저, `e`라는 포인터가 차단된 thread 리스트인 `blocked_list`의 첫 번째 요소를 가리키도록 한다. 이후, while 루프를 사용해 `blocked_list`의 끝까지 탐색하며 각 thread의 `wakeup_tick` 값과 현재 `ticks` 값을 비교한다.

만약 `e`가 가리키는 thread의 `wakeup_tick`이 `ticks`보다 작거나 같다면, 해당 thread를 깨워야 한다. 이를 위해 먼저 `list_remove` 함수를 사용해 해당 thread를 `blocked_list`에서 제거하고, `thread_unblock` 함수를 호출해 thread의 상태를 `THREAD_READY`로 변경한 후, 마지막으로 `swap_running_thread`를 호출하여 thread가 깨어났을 때, 다른 threads와의 priority 경쟁을 고려해 적절히 스케줄링되도록 한다. 반대로, thread를 깨울 필요가 없는 경우에는 `list_next` 함수를 사용하여 `e`가 `blocked_list`의 다음 thread를 가리키도록 한 후, 위의 과정을 반복한다. 이때, `list_remove`는 `~lib/kernel/list.c`에 구현되었으며 아래와 같이 입력받은 `list_elem`을 해당 리스트에서 제거하고 입력받은 `list_elem`의 `next`에 연결된 `list_elem->next`를 반환하므로 `thread_awake`는 현재 tick에서 awake되어야 하는 thread를 blocked list에서 제외하고 다음 thread에 대해서 검사를 이어나갈 수 있도록 한다. 즉 `thread_awake` 함수는 매 tick마다 실행되어 해당 tick에 awake되어야 하는 모든 thread를 unblock하는 conceptual한 alarm clock의 기능을 구현한다.

```
struct list_elem *
list_remove (struct list_elem *elem)
{
    ASSERT (is_interior (elem));
    elem->prev->next = elem->next;
    elem->next->prev = elem->prev;
    return elem->next;
}
```

`thread_awake` 함수 내에서 일어나는 작업들은 다른 thread나 인터럽트가 공유 자원에 동시에 접근하더라도 큰 문제가 발생하지 않기 때문에 원자적으로 이루어질 필요가 없다. 따라서 이 함수에서는 인터럽트를 비활성화하지 않았다.

```
void
thread_awake(int64_t ticks)
{
    struct list_elem *e = list_begin (&blocked_list);

    while (e != list_end(&blocked_list)){
        struct thread *t = list_entry(e, struct thread, elem);
        if(t->wakeup_tick <= ticks){
            e = list_remove(e);
            thread_unblock(t);
            swap_running_thread();
        }else{
            e = list_next(e);
        }
    }
}
```

위의 두 함수를 이용하면 기존 코드를 수정할 수 있다. 먼저, `timer_sleep` 함수에서 기존의 코드는 while 루프를 사용하여 thread가 깨어날 시간을 반복적으로 확인하였으나 수정된 코드에서는 busy waiting을 피하기 위해 `thread_sleep` 함수를 호출하여 thread를 차단하고 `blocked_list`에 추가하는 방식으로 변경하였다.

```
void
timer_sleep (int64_t ticks)
{
    int64_t start = timer_ticks ();
    int64_t end = start + ticks;
    thread_sleep(end);
}
```

또한, `timer_interrupt` 함수에서 `ticks` 값이 증가할 때마다 `thread_awake` 함수를 호출하여, 차단된 thread의 `wakeup_tick` 값을 확인하고 조건을 충족하는 경우 해당 thread를 다시 `THREAD_READY` 상태로 변환한 후 `ready_list`에 추가하는 과정을 수행한다.

```
static void
timer_interrupt (struct intr_frame *args UNUSED)
{
    ticks++;
    thread_tick ();
    thread_awake(ticks);
}
```

Difference from design report

Alarm clock 의 구현은 기존의 디자인 레포트에서 제시한 내용과 차이가 없다. 초기 설계 대로 `thread_block`과 `thread_unblock`을 사용한 방식을 적용했으며, 전체적인 구조와 기능은 설계 단계에서 계획한 대로 구현되었다. 그러나 `thread_sleep` 과 `thread_awake` 함수를 정의하는 과정에서 `wakeup_tick` 이나 `blocked_list` 와 같은 추가적인 변수와 리스트를 추가적으로 정의하여 구현한 부분이 추가되었다.

Limitations

위의 구현방식에서 `thread_block`과 `thread_unblock` 을 사용하는 과정에서 만약 여러 thread가 위의 과정을 반복한다면 context switching 비용이 증가하여 성능이 저하될 수 있을 것 같다는 생각을 하였다.

또한, 그러나 `thread_block` 코드 주석에서 다음과 같은 내용을 언급하고 있다:

```
/** Puts the current thread to sleep. It will not be scheduled
    again until awoken by thread_unblock().

    This function must be called with interrupts turned off. It
    is usually a better idea to use one of the synchronization
    primitives in synch.h. */
```

이 주석은 `thread_block`과 `thread_unblock`을 사용하는 것보다 synchronization primitives를 사용하는 것이 더 적합하다고 설명하고 있다. 그 이유를 찾아본 결과, synchronization primitives (i.e. semaphore, lock, condition variables) 를 사용하면 더 높은 수준의 추상화(abstraction)를 제공하기 때문이다. 또한, synchronization primitives 를 사용하면 인터럽트를 비활성화하는 등의 명시적인 보호 없이도 race condition 이 발생하는 것을 방지할 수 있다고 한다.

Discussion

이번 Alarm Clock 과제를 통해 시스템 자원의 활용도를 높이기 위해 busy waiting 방식이 아닌 thread block과 unblock 방법을 직접 구현하면서 스케줄링의 작동 원리에 대해 배울 수 있었다. 또한, 여러 thread가 동시에 shared resource 에 접근할 때 발생하는 race condition 에 대해 이해하게 되었으며, 이를 방지하기 위한 방법으로 interrupt 를 disable하는 방법과 synchronization primitives 를 사용하는 방법을 배울 수 있었다.

2. Priority Scheduling

Implementation & Improvement from the previous design

Design report에서 설명한 것처럼, 현재의 구현 방식은 priority와 상관없이 round-robin 방식으로 스케줄링이 이루어지고 있다. 이번 과제에서는 이 방식을 priority를 고려한 스케줄링 방식으로 수정하였다.

이를 위해 priority에 따른 정렬 과정이 필요하며, 두 가지 접근 방식이 가능하다. 첫 번째는 thread를 `ready_list` 에 추가할 때 priority에 따라 정렬하는 방식이고, 두 번째는 CPU가 다른 thread로 전환될 때 `ready_list`에서 다음 실행될 thread를 선택하면서 정렬하는 방식이다. 후자의 경우, 매번 context switching이 발생할 때마다 정렬을 수행해야 하므로 효율적이지 않다고 판단하여, 전자의 방식으로 구현하였다.

그렇다면 언제 thread가 `ready_list`에 추가되는지를 이해하기 위해서는 `thread_unblock` 함수와 `thread_yield` 함수를 살펴보면 된다. `thread_unblock` 함수는 블록된 thread, 즉 `THREAD_BLOCKED` 상태에 있는 thread를 `THREAD_READY` 상태로 변경하고, 이를 `ready_list`에 추가하는 함수이자 위의 Alarm Clock 과제에서 설명하였기

때문에 기타 설명을 생략하겠다. `thread_yield`는 현재 실행 중인 `thread`가 자발적으로 CPU 점유를 다른 실행 가능한 `thread`에게 넘길 때 호출된다. 즉 `thread_yield`는 자기 자신을 `ready_list`에 insert 한다.

기존의 구현에서는 `list_push_back` 함수를 사용해 `ready_list`의 가장 뒤에 `thread_elem`을 추가했지만, 수정된 구현에서는 `list_insert_ordered` 함수를 사용하여 `priority`에 맞게 정렬후 삽입을 수행하였다. 아래의 `list_insert_ordered` 함수는 조건에 맞는 위치를 찾아 `list_insert` 함수를 호출하여 해당 위치에 요소를 삽입하는 역할을 한다.

```
void
list_insert_ordered (struct list *list, struct list_elem *elem,
                    list_less_func *less, void *aux)
{
    struct list_elem *e;

    ASSERT (list != NULL);
    ASSERT (elem != NULL);
    ASSERT (less != NULL);

    for (e = list_begin (list); e != list_end (list); e = list_next (e))
        if (less (elem, e, aux))
            break;
    return list_insert (e, elem);
}
```

이러한 `list_insert_ordered`가 올바르게 `thread`의 `priority`를 비교 정렬하여 삽입을 수행할 수 있게하기 위해서는 두 `list_elem`의 `thread`간의 `priority`의 대소를 비교하여 boolean으로 return하는 비교함수를 정의하여야 한다. 따라서 위에서 설명한 바와 같이 `priority`를 비교하기 위해 `cmp_pri` 함수를 정의하였다. 이 함수는 두 개의 `thread`, `x`와 `y`에 해당하는 `list_elem`을 입력받아 그 둘의 `priority`를 비교한 다음, 만약 `x`의 `priority`가 `y`의 `priority`보다 크다면 `true`를 반환하고, 그렇지 않다면 `false`를 반환한다.

```
bool
cmp_pri(struct list_elem *a, struct list_elem *b, void *aux UNUSED)
{
    struct thread *x = list_entry(a, struct thread, elem);
    struct thread *y = list_entry(b, struct thread, elem);
    return x->priority > y->priority;
}
```

이렇게 정의한 `cmp_pri` 함수를 `list_insert_order` 함수의 비교 함수로 사용하여 `thread_unblock`과 `thread_yield` 함수에 구현하였으며, 그 결과는 다음과 같다.

```
void
thread_unblock (struct thread *t)
{
    ...

    ASSERT (t->status == THREAD_BLOCKED);
}
```

```

/*****/
list_insert_ordered(&ready_list, &t->elem, cmp_pri, 0);
/*****/

...
}

void
thread_yield (void)
{
    ...
    if (cur != idle_thread)
        /*****/
        list_insert_ordered(&ready_list, &cur->elem, cmp_pri, 0);
        /*****/
    ...
}

```

ready_list에 list_elem을 삽입하는 두 함수 모두에 **list_insert_ordered**가 사용되었으므로 ready_list는 모든 입력에 대해 priority가 정렬됨이 보장된다. 따라서 위의 방식대로 구현하면 ready_list의 head에 있는 thread가 선택되어 CPU를 점유할 때, 저절로 가장 priority가 높은것이 되도록 선택된다. (목표 #1) 그 후, 현재 실행 중인 thread와 ready_list에서 가장 앞에 있는 (즉, 가장 priority가 높은) thread의 priority를 비교해야 한다.

swap_running_thread 함수는 is_running_valid 함수가 참일 경우 아무 작업도 수행하지 않지만, 거짓일 경우 swap_running_thread_helper 함수를 호출한다. is_running_valid 함수는 현재 thread가 idle 상태인지, 그리고 ready_list가 비어 있는지를 확인하는 함수이다. 두 조건 중 하나라도 참이면 true를 반환하여 swapping을 방지하지만, 두 조건이 모두 거짓이면 (즉, 현재 thread가 idle이 아니고 ready_list에 하나 이상의 thread가 있을 경우) swap_running_thread_helper 함수를 호출하게 된다.

```

bool
is_running_valid(void)
{
    bool flag = (thread_current() == idle_thread) || list_empty(&ready_list);
    return flag;
}

void
swap_running_thread(void)
{
    if(is_running_valid()){
        return;
    }else{
        swap_running_thread_helper();
    }
}

```

앞서 설명한 swap_running_thread_helper 함수는 현재 실행 중인 thread와 대기 중인 thread 중에서 ready_list의 가장 앞에 있는 thread(즉, priority가 가장 높은 thread)의 priority를 비교하고, thread_yield 함수를 통해 CPU를 넘겨줄 수 있도록 하는 함수이다. 현재 실행 중인 thread는 c 포인터가 가리키고, ready_list의 가장 앞에 있는

thread는 r 포인터가 가리키도록 한다. 이후 두 thread의 priority를 비교한 다음, r의 priority가 더 높다면 thread_yield 함수를 호출하여 CPU를 넘겨주도록 하였다.

이때 ready_list에서 priority가 가장 높은 thread를 r 포인터가 가리킬 수 있는 이유는, 앞서 thread를 ready_list에 삽입할 때 이미 priority에 따라 정렬이 이루어졌기 때문이다. 따라서 ready_list의 가장 앞에 있는 thread가 항상 가장 높은 priority를 가지게 된다.

이 과정에서 thread_yield 함수를 호출하게 되는데, 외부 인터럽트가 발생한 상황에서 이를 호출하면 커널 패닉(kernel panic)이 발생할 수 있었다. 이를 방지하기 위해 intr_context() 함수를 사용하여, 외부 인터럽트 발생 시에는 문맥 교환(context switching)이 이루어지지 않도록 수정하였다.

```
void
swap_running_thread_helper(void)
{
    struct thread *c = thread_current();
    struct thread *r = list_entry(list_front(&ready_list), struct thread, elem);
    if( !intr_context() && c->priority < r->priority){
        thread_yield();
    }
}
```

이제 두 thread의 priority를 비교하고, 필요한 경우 CPU를 넘겨주는 함수를 구현했으므로, 이를 기존 코드에 추가하기만 하면 된다. 이러한 비교 과정이 필요한 경우는 두 가지가 있다. 첫 번째는 thread_create를 통해 thread가 새롭게 생성되어 ready_list에 추가될 때이다. 새롭게 생성된 thread의 priority가 가장 높을 수 있으므로, 이 과정을 실행해야 한다. 두 번째는 thread_set_priority 함수에서 현재 thread의 priority가 새롭게 정의될 때이다. 이 경우에도 priority가 변했으므로 위의 과정을 반복해야 한다.

따라서 thread_create 함수와 thread_set_priority 함수의 마지막 부분에 swap_running_thread 함수를 추가하였다.

```
tid_t
thread_create (const char *name, int priority,
               thread_func *function, void *aux)
{
    ...
    /* Add to run queue. */
    thread_unblock (t);
    /*****/
    swap_running_thread();
    /*****/

    return tid;
}

void
thread_set_priority (int new_priority)
{
    thread_current ()->priority = new_priority;
```



```

/*****/
swap_running_thread();
/*****/
}

```

`swap_running_thread`는 일종의 `thread_current()`가 유효한, 다시말해 `priority`가 최대임을 보장할 수 있도록 하는 함수이고 `thread`의 생성과 `priority value`의 변경이후 해당 함수가 매번 호출되므로 전체 과정에서 가장 높은 `priority`의 `thread`가 실행되도록 보장한다. 위 코드를 수정한 후, 두 번째 목표였던 현재 실행 중인 `thread`보다 `ready_list`의 `thread`가 더 높은 `priority`를 가질 경우 CPU를 넘겨주는 기능이 구현되었다.

다음으로는 `synchronization primitives` 간의 스케줄링을 구현해야 한다. 이번 과제에서는 `lock`, `conditional variable`, 그리고 `semaphore`에서 모두 스케줄링을 적용해야 한다. Design report에서 설명한 것처럼, `semaphore`를 이용해 `lock`을 구현하면, `semaphore`가 초기 값으로 `value = 1`을 가진다는 점을 제외하고는 기존의 `semaphore`와 동일하게 동작하므로 추가적인 함수 수정이나 구현이 필요 없다. 따라서, `semaphore`와 `conditional variable`에 관련된 함수만 수정하거나 추가로 구현하면 된다.

`sema_down`과 `sema_up` 함수의 작동을 살펴보면, `list_push_back`을 통해 `semaphore`의 `waiting_list`에 현재 `thread`를 추가하고, `list_pop_front`를 통해 `waiting_list` 중 가장 앞에 있는 `thread`를 `ready_list`에 추가하는 방식으로 동작한다는 것을 알 수 있다.

앞선 과정과 유사하게, `waiting_list`도 정렬된 상태로 유지하기 위해 `list_push_back` 대신 `list_insert_ordered`를 사용하여 `thread`를 `semaphore`의 `waiting_list`에 추가한다. 이때, 앞서 정의한 `cmp_pri` 함수를 사용하여 `waiting_list` 내 `threads`의 `priority`를 비교하고 정렬할 수 있다.

```

void
sema_down (struct semaphore *sema)
{
    ...
    while (sema->value == 0)
    {
        /*****/
        list_insert_ordered(&sema->waiters, &thread_current()->elem, cmp_pri, 0);
        /*****/
        thread_block ();
    }
    ...
}

```

또한, `sema_up`에서는 `waiting_list`에 있는 동안 `priority`가 변경될 수 있으므로, 먼저 `list_sort` 함수를 사용하여 `waiting_list` 내 `threads`를 정렬한 후, `list_pop_front`를 통해 가장 `priority`가 높은 `thread`를 `ready_list`에 추가하도록 하였다. 그리고 현재 실행 중인 `thread`와 방금 `ready_list`에 추가된 `thread` 간의 `priority`를 비교하기 위해 앞서 구현한 `swap_running_thread` 함수를 호출하였다.

```

void
sema_up (struct semaphore *sema)
{
    enum intr_level old_level;

```

```

    ASSERT (sema != NULL);

    old_level = intr_disable ();
    if (!list_empty (&sema->waiters)){
        /*****/
        list_sort(&sema->waiters, cmp_pri, 0);
        /*****/
        thread_unblock (list_entry (list_pop_front (&sema->waiters), struct thread,
elem));
    }
    sema->value++;
    /*****/
    swap_running_thread();
    /*****/
    intr_set_level (old_level);
}

```

다음으로, conditional variable 함수를 살펴보면, 이 함수 역시 list_push_back과 list_pop_front를 사용하여 conditional variable의 waiting_list에 semaphore의 요소를 추가하는 방식으로 동작한다. 여기서 conditional variable의 waiting_list는 semaphore들의 리스트이며, 각 semaphore는 이미 앞서 설명한 방식으로 정렬되었기 때문에, conditional variable의 waiting_list만 정렬하면 가장 priority가 높은 thread를 결정할 수 있다. 이를 위해 list_insert_ordered 함수를 사용하여 정렬을 진행한다.

다만, 여기서 중요한 차이점은, 이전 과정에서는 cmp_pri를 사용하여 threads 간의 priority를 비교했지만, 이번에는 semaphore들 간의 priority를 비교해야 한다는 것이다. cmp_sema_pri는 get_sema_pri를 사용하여 각각의 list_elem에 속한 semaphore의 waiting_list에서 가장 높은 priority 값을 가져와 비교한다. 만약 a의 priority가 더 높으면 참을 반환하고, 그렇지 않으면 거짓을 반환하도록 구현되었다.

get_sema_pri 함수는 먼저, 매개변수로 받아온 e를 semaphore_elem 구조체로 변환한 후, 해당 구조체 내부의 semaphore의 waiting_list를 l이 가리키도록 한다. 그 후, list_begin 함수를 사용하여 waiting_list의 첫 번째 thread(즉, priority가 가장 높은 thread)를 node가 가리키도록 하고, 이 node를 thread 구조체로 변환한다. 이를 통해 해당 semaphore의 waiting_list에서 가장 priority가 높은 thread인 t의 priority를 반환한다.

```

int
get_sema_pri(struct list_elem *e)
{
    struct semaphore_elem *s = list_entry(e, struct semaphore_elem, elem);
    struct list *l = &(s->semaphore.waiters);
    struct list_elem *node = list_begin(l);
    struct thread *t = list_entry(node, struct thread, elem);
    int sema_pri = t->priority;

    return sema_pri;
}

bool
cmp_sema_pri(struct list_elem *a, struct list_elem *b, void *aux UNUSED)
{
    bool flag = get_sema_pri(a) > get_sema_pri(b);
}

```

```

    return flag;
}

```

위에서 만든 비교함수를 `list_insert_ordered` 함수에 추가하여 `cond_wait` 함수에 `list_push_back` 대신 사용하면 semaphore 들이 sorted 된 상태로 conditional variable 의 waiting list 에 추가되게 된다.

```

void
cond_wait (struct condition *cond, struct lock *lock)
{
    ...
    sema_init (&waiter.semaphore, 0);

    /*****/
    list_insert_ordered(&cond->waiters, &waiter.elem, cmp_sema_pri, 0);
    /*****/

    lock_release (lock);
    ...
}

```

`cond_signal` 을 할 때도 마찬가지로 waiting_list 에 있는 사이에 priority 변화가 생겼을 수도 있으니 `list_sort` 함수를 통해 다시 정렬을 맞춰주는 과정이 필요하다.

```

void
cond_signal (struct condition *cond, struct lock *lock UNUSED)
{
    ...
    if (!list_empty (&cond->waiters)) {
        /*****/
        list_sort(&cond->waiters, cmp_sema_pri, 0);
        /*****/
        sema_up (&list_entry (list_pop_front (&cond->waiters),
                                     struct semaphore_elem, elem)->semaphore);
    }
}

```

여기까지 세번째 목표인 synchronization primitives 사이의 CPU yield 가 구현되었다. 하지만 이렇게 구현하고 나면 priority inversion 문제가 발생한다. 해당 문제는 높은 priority를 가진 thread 가 낮은 priority 를 가진 thread 때문에 blocked 된 상태를 말한다. 즉, 낮은 priority 를 가진 thread 가 lock 을 보유하고 있어, 높은 priority 를 가진 thread 가 실행이 안되는데 cpu 는 더 높은 priority 를 가진 thread 가 있기 때문에 낮은 priority 를 가진 thread 가 실행이 되지 않는 상태를 말한다.

위의 문제를 해결하기 위해서는 multiple priority donation 을 구현해야한다. priority donation 은 낮은 priority 를 가진 thread 가 lock 을 release 할 수 있도록 높은 priority 를 가진 thread 가 자신의 priority 를 "donate" 해주는 것을 말한다. pintos document 에 의하면 총 8번까지 donation 이 이루어지는 것이 이상적이라고 설명되어 있어 이번 과제에서도 8 번까지 donation 이 되는 것을 목표로 구현하였다.

이번 과제에서는 priority 상속을 관리하기 위한 inherit_manager 구조체를 새롭게 정의하였다. 이 구조체는 thread 간 priority 상속 과정에서 필요한 정보를 관리하는 역할을 한다. 먼저, original_pri는 thread의 원래 priority를 저장하여 priority 상속이 해제될 때 이를 복원할 수 있게 한다. current_pri는 priority를 상속받았을 때 임시로 부여된 새로운 priority를 저장하며, 스케줄링에 반영된다.

또한, inheritor_list는 priority를 기부한 threads의 연결 리스트로, 여러 thread가 priority를 기부할 경우 이들을 관리하고 추적하는 역할을 한다. inheritor는 현재 thread가 priority를 상속해줄 대상 thread의 리스트에 자신을 추가할 때 사용하는 list_elem 구조체로 정의되었으며, 마지막으로 target_lock은 현재 thread가 대기 중인 특정 lock을 가리키는 포인터로, priority 상속 과정에서 어떤 lock이 관련되어 있는지 추적하는 데 사용된다.

```
struct inherit_manager
{
    int original_pri; /* 원래 thread의 pri */
    int current_pri; /* 현재 상속받아 임시로 부여된 pri */
    struct list inheritor_list; /* 나한테 상속해준 thread 연결리스트 */

    struct list_elem inheritor; /* 내가 상속해주는 thread의 상속리스트에 적을 이름표 */
    struct lock *target_lock; /* 내가 대기중인 lock */
};
```

inherit_manager 구조체를 정의한 후, 이를 thread 구조체에 manager라는 이름으로 선언하였다. 이렇게 새로운 구조체를 정의한 후, thread가 생성될 때 호출되는 init_thread 함수에서 해당 구조체를 초기화해주었다. 이 초기화 과정은 인터럽트가 비활성화된 상태에서 진행되며, 먼저 thread의 현재 priority를 manager 구조체의 original_pri 필드에 저장하고, thread가 대기 중인 lock이 없으므로 target_lock을 NULL로 설정한다. 또한, manager의 inheritor_list를 초기화하기 위해 list_init 함수를 사용하였다. list_init 함수는 리스트의 시작과 끝을 초기화하여, 리스트가 비어 있는 상태에서 시작하도록 설정하는 역할을 한다.

```
struct thread
{
    /*******/
    struct inherit_manager manager;
    /*******/
    ...
}

static void
init_thread (struct thread *t, const char *name, int priority)
{
    ...
    t->manager.original_pri = priority;
    t->manager.target_lock = NULL;
    list_init(&(t->manager.inheritor_list));
    ...
}
```

이제 priority donation을 관리할 수 있는 구조체 정의와 초기화가 완료되었으므로, 이를 실제 함수 내에서 사용해야 한다. 먼저 lock_acquire 함수를 살펴보면, sema_down 함수 이전에는 thread가 아직 lock을 획득하지 못한 상태이며, sema_down을 호출한 이후에 lock을 획득하게 된다. 따라서, sema_down 함수가 호출되기 전에 현재 lock을 보유한 thread에게 priority를 기부해야 한다. 이를 위해 먼저 lock->holder가 NULL인지 확인하여, 현재 lock을 소유한 thread가 있는지 검사한다. 만약 lock->holder가 NULL이라면, thread가 lock을 획득할 수 있는 상태이므로 priority donation 과정이 필요 없다. 그러나 lock->holder가 NULL이 아니라면, priority donation 과정을 수행해야 한다.

여기서 중요한 점은, 이미 다른 thread가 lock 을 보유한 상태에서 lock_acquire가 호출되었다면, 현재 실행 중인 thread가 lock 을 보유한 thread보다 더 높은 priority를 가지고 있다는 것을 의미한다. 그 이유는, 앞서 구현한 priority scheduling 에 따르면 항상 가장 priority가 높은 thread가 실행되기 때문이다.

따라서, priority donation 과정에서는 먼저 현재 thread의 target_lock 필드에 해당 lock 을 할당한다. 이후 inheritor_list에 현재 thread를 추가하고, inherit_pri 함수를 실행하여 priority donation을 처리한다.

```
void
lock_acquire (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (!intr_context ());
    ASSERT (!lock_held_by_current_thread (lock));

    /*****/
    struct thread *t = thread_current();
    if(lock->holder != NULL){
        t->manager.target_lock = lock;
        list_insert_ordered(&lock->holder->manager.inheritor_list, &t-
>manager.inheritor, cmp_inheritor_pri, NULL);
        inherit_pri();
    }
    /*****/

    sema_down (&lock->semaphore);
    lock->holder = thread_current ();
}
```

위의 list_insert_ordered 함수에서 사용된 비교 함수 `cmp_inheritor_pri`는 두 매개변수 x와 y 각각의 priority(priority)를 비교하여, x의 priority가 더 크면 true를 반환하고 그렇지 않으면 false를 반환하도록 한다. 이 함수는 리스트에서 thread의 priority를 기준으로 정렬할 때 사용된다. 이 함수에서 list_entry는 각각의 리스트 요소 x와 y를 struct thread로 변환하고, 그 thread의 manager.inheritor 필드를 통해 priority를 가져와 비교하는 역할을 한다.

```
bool
cmp_inheritor_pri(struct list_elem *x, struct list_elem *y, void *aux UNUSED)
{
    return list_entry(x, struct thread, manager.inheritor)->priority > list_entry(y,
struct thread, manager.inheritor)->priority;
}
```

inherit_pri 함수는 앞서 설명한 것처럼 최대 8번까지 priority donation 이 이루어지도록 구현되었다. 또한, 이 함수는 nested donation 방식으로 구현되었는데, 이는 가장 높은 priority의 thread가 그보다 낮은 priority를 가진 thread에게 priority를 기부하고, 이 과정을 반복함으로써 결국 가장 높은 priority가 가장 낮은 priority를 가진 thread에까지 전달되도록 하는 방식이다. 아래 코드를 보면 for 루프 안에서 최대 8번 이내에 target_lock을 보유한 thread에 도달할 때까지 lock 을 소유한 thread에게 현재 thread의 priority를 기부하는 방식으로 구현되었다. 만약 target_lock이 NULL이면 더 이상 priority donation 이 필요 없으므로 for 루프를 종료(break)하도록 하였다.

```
void
inherit_pri(void)
{
    int i;
    struct thread *t = thread_current ();

    for (i = 0; i < 8; i++){
        if (!t->manager.target_lock) break;
        struct thread *holder = t->manager.target_lock->holder;
        holder->priority = t->priority;
        t = holder;
    }
}
```

위의 과정에서 priority donation 까지 끝마친 후, sema_down 함수가 호출되고 반환된 이후에는 현재 thread가 lock 을 획득한 상태이므로, target_lock을 NULL로 설정하고, lock->holder를 현재 thread로 지정하는 과정이 필요하다. sema_down이 성공적으로 실행되면 현재 thread가 해당 lock 을 소유하게 되므로, 더 이상 다른 thread에게 priority donation 이 필요하지 않다. 따라서 target_lock을 NULL로 설정하여 priority donation 이 중단되도록 한다.

```
void
lock_acquire (struct lock *lock)
{
    ...
    sema_down (&lock->semaphore);
    /*****/
    t->manager.target_lock = NULL;
    /*****/
    lock->holder = thread_current ();
}
```

위의 과정을 통해 lock 을 acquire 할 때의 priority donation 과정을 성공적으로 구현하였다. 이렇게 donated 된 priority를 lock 이 release 될 때 원래대로 돌려놓는 코드를 추가하였다. 기존의 lock_release 코드에서는 lock 의 holder 를 null 로 설정하고 sema_up 을 호출하게 된다. 하지만 이제는 sema_up 함수를 호출하기 전에 priority 를 donation 해준 thread 들의 priority 를 재설정하고 donation list 에서 제거하는 과정이 추가되어야 한다.

priority 를 재설정하고 donation list 에서 제거시 필요한 동작을 구현하기 위해 restore_pri와 release_helper라는 두 가지 함수를 추가하였다. 먼저, restore_pri 함수는 thread의 priority를 원래 상태로 복원하고, 필요에 따라 상속된 priority를 유지하는 역할을 한다.

restore_pri 함수는 먼저 현재 실행 중인 thread를 가져와, 그 thread의 priority를 original_pri로 되돌린다. 이후, thread의 inheritor_list가 비어 있지 않다면, 즉 priority를 기부한 thread가 있을 경우, list_sort 함수를 사용하여 inheritor_list를 priority에 따라 정렬한다. 이때, 위에서 정의한 cmp_inheritor_pri 함수를 통해서 priority를 비교한다. 그 후, inheritor_list에서 제일 앞에 있는, 즉 가장 높은 priority를 가진 thread를 가져와, 해당 thread의 priority가 현재 thread의 priority보다 높으면 현재 thread의 priority를 기부된 priority로 업데이트한다.

```
void
restore_pri(void)
{
    struct thread *t = thread_current();
    t->priority = t->manager.original_pri;

    if (!list_empty (&t->manager.inheritor_list)) {
        list_sort (&t->manager.inheritor_list, cmp_inheritor_pri, 0);

        struct thread *nxt = list_entry (list_front (&t->manager.inheritor_list),
        struct thread, manager.inheritor);
        if (nxt->priority > t->priority)
            t->priority = nxt->priority;
    }
}
```

release_helper 함수는 특정 lock 을 해제할 때 priority donation 과 관련된 threads 의 상태를 정리하는 역할을 한다. 즉, 해당 lock 에 대해 priority를 기부했던 threads 를 inheritor_list에서 제거하여 priority donation 상태를 업데이트하고자 한다. 먼저 현재 thread의 inheritor_list를 순회하며, 각 thread의 target_lock이 인자로 전달된 lock 과 일치하는지 확인한다. 만약 일치하는 thread가 있으면, 해당 thread를 inheritor_list에서 제거하여 더 이상 해당 thread가 lock 과 관련된 priority donation 을 유지하지 않도록 한다. 이렇게 함으로써, 특정 lock 이 해제될 때 관련된 priority donation 관계를 정리하고, priority가 적절히 관리될 수 있도록 한다.

```
void
release_helper(struct lock *l)
{
    struct list_elem *e;
    struct thread *t = thread_current();

    for (e = list_begin (&t->manager.inheritor_list); e != list_end (&t->manager.inheritor_list); e = list_next (e)){
        struct thread *t = list_entry (e, struct thread, manager.inheritor);
        if (t->manager.target_lock == l)
            list_remove (&t->manager.inheritor);
    }
}
```

위에서 구현한 함수들을 사용하여 lock_release 함수를 수정하였다. 먼저 release_helper를 호출하여, 현재 thread가 더 이상 해당 lock 에 대해 priority를 기부받지 않도록 inheritor_list에서 관련 thread를 제거한다. 이후 restore_pri 함수를 호출하여, thread의 priority를 원래 priority로 복원하거나, 만약 priority를 기부한 다른 thread가 남아 있다면 그 thread의 priority를 유지하도록 하였다. 이러한 과정을 통해 priority donation 과 관련

된 상태를 적절히 정리한 후, lock->holder를 NULL로 설정하고, sema_up을 호출하여 lock 을 대기 중인 다른 thread에게 lock 을 넘긴다.

```
void
lock_release (struct lock *lock)
{
    ASSERT (lock != NULL);
    ASSERT (lock_held_by_current_thread (lock));

    /*****/
    release_helper(lock);
    restore_pri();
    /*****/

    lock->holder = NULL;
    sema_up (&lock->semaphore);
}
```

추가적으로, 현재 실행 중인 thread의 priority가 변경될 가능성이 있다. 이는 priority donation 에 의해 현재 실행 중인 thread의 priority가 상속받은 priority가 아닌, 새로운 priority로 변경될 수 있기 때문이다. priority가 변경되면 스케줄링에 영향을 미칠 수 있으므로, priority 변경 시 이를 적절히 처리해 주어야 한다.

아래의 코드를 보면, 첫 번째로, thread의 original_pri를 새로 설정된 priority 값으로 업데이트한다. 이 과정은 priority donation 이 발생했을 때, 원래 thread의 priority를 기억하고 복원하는 데 필요하다.

두 번째로, restore_pri() 함수는 priority donation 이 이루어졌을 때, 현재 thread의 priority를 복원하거나, 상속 받은 priority가 있다면 이를 유지하도록 한다. 이를 통해 priority가 적절하게 관리되고, 기부된 priority가 있는 경우 thread의 priority가 올바르게 설정된다.

마지막으로 swap_running_thread()는 앞에서 추가된 함수로, priority가 변경된 이후 스케줄러가 가장 priority가 높은 thread를 실행하도록 thread를 교체하는 역할을 한다. priority가 변경되면, 더 높은 priority를 가진 thread가 있을 경우 현재 실행 중인 thread를 멈추고 새로운 thread를 실행시키도록 한다.

```
void
thread_set_priority (int new_priority)
{
    thread_current ()->priority = new_priority;

    /*****/
    thread_current ()->manager.original_pri = new_priority;
    restore_pri();
    /*****/

    swap_running_thread(); //앞에서 추가 됨
}
```

Difference from design report

Design report에서는 단순히 linked list로 priority를 기부한 threads 를 donation_list에 순서대로 추가하고, 나중에 이를 역순으로 처리하면 된다고 설명하였다. 그러나 실제로 구현하면서, 이 과정이 생각보다 복잡하다는 것을 깨달았다.

우선, priority donation 의 중첩(nested donation) 문제를 처리하는 것이 예상보다 복잡했다. 단순히 priority를 기부한 thread를 리스트에 추가하는 것만으로는 충분하지 않았다. 한 thread가 여러 번 기부받는 경우나, 여러 thread가 동일한 thread에 priority를 기부하는 경우를 모두 처리해야 했기 때문이다. 이 과정에서 각 thread가 어떤 lock 을 기다리고 있는지 추적해야 했고, lock 을 소유한 thread의 priority가 계속해서 갱신될 수 있는 구조를 만들어야 했다.

또한, priority donation 이 끝났을 때, 기부된 priority를 원래대로 복원하는 과정 역시 복잡했다. 단순히 리스트에서 priority를 되돌리는 것이 아니라, 남아 있는 기부된 priority들 중 가장 높은 priority를 찾아야 했다. 이를 위해 기부 리스트를 정렬하고, 복원 과정에서 가장 높은 priority를 가진 thread를 적절히 선택해야 했다. 특히, 여러 thread가 기부를 받거나 주는 상황에서 thread 간의 상호작용이 복잡해졌으며, priority donation 이 일어나는 깊이(nested depth)를 관리해야 했다.

또한, priority donation 이 다양한 시점에서 발생할 수 있다는 점도 문제를 복잡하게 만들었다. lock 을 요청하는 시점에서 뿐만 아니라, lock 을 해제하거나 priority를 변경하는 경우에도 기부된 priority를 처리해야 했기 때문에, 각 함수에서 이러한 기부 상태를 적절하게 반영할 수 있도록 코드를 수정해야 했다. 이를 통해 release_helper와 restore_pri 같은 보조 함수를 추가로 구현하게 되었고, 이 함수들이 서로 연계되어 작동하도록 구조를 만들어야 했다.

Limitations

running thread와 ready_list에서 가장 우선순위가 높은 스레드를 비교하여 필요한 경우 thread_yield를 호출하려 할 때, kernel panic 오류가 발생하였다. 문제를 조사한 결과, 내부 interrupt 뿐만 아니라 외부 interrupt 도 예외 처리가 필요하다는 것을 알게 되었다. 내부 interrupt는 다른 함수에서 intr_disable을 통해 비활성화하고 있었지만, 외부 interrupt가 발생하는지 확인하기 위해 intr_context를 활용하여 외부 인터럽트가 있는 경우에는 context switching을 하지 않도록 처리해야 하였다. 여기서 외부 인터럽트는 주로 I/O 장치와 같은 하드웨어에서 발생하는 인터럽트를 의미한다.

또한, lock을 semaphore로 구현할 때, sema_up에서 waiting_list에 스레드가 있는 동안 우선순위가 변경될 수 있기 때문에 이를 반영하기 위해 list_sort를 다시 수행해야 하였다. 초반에는 waiting_list에 스레드를 삽입할 때만 정렬된 상태로 추가하였으나, 그 결과가 예상과 달랐다. 마찬가지로 conditional variable에서도 semaphore의 우선순위를 재정렬하는 작업이 필요하였다.

마지막으로, 우선순위 상속(priority donation) 과정에서도 nested donation을 이론적으로 구현하는 것뿐만 아니라, 실제로 donation list를 만들어 최대 8번까지 우선순위 상속을 한 후 다시 원래의 우선순위로 복원하는 과정을 구현하는 데 코드의 복잡성 때문에 어려움이 있었다.

책을 읽어보았을 때, semaphore를 사용하지 않고 lock과 conditional variable을 구현하는 것이 권장된다고 하였다. 그 이유는 먼저, 명시적인 타입을 사용함으로써 lock과 conditional variable을 사용하는 부분이 semaphore 호출과 혼동되지 않아 코드의 명확성을 높이는 데 도움이 되기 때문이다. 또한, semaphore를 사용해 conditional variable을 구현하면 더 이상 conditional variable이 stateless하지 않게 되는데, 이는 semaphore가 값을 유지하기 때문이다. semaphore의 값에 따라 동작이 결정되므로, 프로그래머가 객체 상태를 semaphore의 값에 맞추어 신중하게 관리해야 하며, 이로 인해 semaphore 사용이 권장되지 않는다고 한다.

그러나 Pintos가 이러한 권장되지 않는 semaphore를 통한 구현 방식을 선택한 이유는, 설계가 단순하고 효율적이기 때문이다. semaphore는 블로킹/언블로킹 동작을 효과적으로 처리할 수 있기 때문에, lock과 conditional

variable을 semaphore로 쉽게 구현할 수 있는 장점이 있다는 것을 알게되었다.

Discussion

이번 과제를 통해 기존의 round-robin 방식으로 구현되어 있던 스케줄링을 priority를 기반으로 정렬된 상태로 구현하는 방법을 배울 수 있었다. 특히, lock과 conditional variable이 semaphore로 구현되어 있다는 점과, 이러한 구조에서 priority 기반 스케줄링을 유지하기 위해 필요한 수정 사항들을 이해할 수 있었다.

또한, priority inversion 문제에도 배우게 되었다. priority inversion은 시스템에서 낮은 priority의 thread가 lock을 소유하고 있을 때, 더 높은 priority의 thread가 이를 기다리며 블록되는 상황을 의미한다. 이러한 상황은 시스템 성능 저하를 초래할 수 있으며, 이를 해결하기 위한 priority donation을 구현하는 것이 두번째 주제의 어려운 부분이었다.

priority donation을 구현하는 과정에서, 단순히 기부된 priority를 저장하고 관리하는 것이 아닌, 기부된 priority가 nested 되고, 다양한 thread 간에 상호작용할 수 있는 복잡한 상황들을 처리하는 방법을 배울 수 있었다. 특히, priority가 여러 번 기부되는 경우, 기부된 priority들을 정렬하고, 적절히 복원하는 과정이 매우 중요하다는 것을 깨달았다. 이를 위해 restore_pri와 같은 함수를 통해 기부된 priority가 끝난 후에도 시스템이 일관성을 유지할 수 있도록 설계하는 것이 필요했다.

또한, lock 해제 시 priority 복원과 기부된 priority 관리가 어떻게 연계되는지에 대한 이해도 깊어졌다. release_helper 함수와 같이 lock을 해제할 때 관련된 threads의 priority donation 상태를 정리하고, thread의 priority를 원래대로 복원하는 과정을 통해 스케줄링의 복잡한 문제들을 해결할 수 있었다.

3. Advanced scheduler

Implementation & Improvement from the previous design

마지막 과제는 MLFQS (Multi-Level Feedback Queue Scheduler) 스케줄러를 구현하는 것이다. 이전에 구현한 priority queue는 하나의 값(priority)에 의해 CPU 점유 순서를 결정하는데, priority 값이 높은 프로세스는 계속해서 CPU를 점유하고, 낮은 priority를 가진 프로세스는 실행 기회를 얻지 못하는 starvation 문제가 생길 수 있다. 이 경우 시스템의 공정성이 떨어지며, 중요한 작업을 계속 대기하게 되는 상황이 발생할 수 있다. 이러한 문제를 해결하기 위해 새로운 스케줄링 방식인 MLFQS를 구현하였다.

MLFQS에서는 priority에 따라 여러 개의 ready queue가 존재하며, 이것이 multi-level인 이유다. 또한 priority는 timer tick에 따라 지속적으로 조정되는데, 이것이 feedback 메커니즘이다.

Pintos documentation에 따르면, thread_mlfqs라고 불리는 boolean 변수를 통해 현재 어떤 스케줄러를 사용 중인지 알 수 있다. 해당 documentation에서는 새로운 측정값들이 소개되어 있는데, nice, priority, recent_cpu, 그리고 load_avg가 그 예시이다. 각 값에 대한 구현 방법을 아래에서 설명할 것이다.

먼저 nice 값은 해당 값이 높을수록 CPU를 더 적게 점유하게 되며, 다른 thread에게 보다 nice함을, 즉 CPU 사용을 더 잘 양보한다것을 의미한다. 이 값은 -20부터 20까지의 integer 값을 가질 수 있으며, 매 tick마다 다시 계산된다. nice 값이 높을수록 프로세스는 낮은 priority를 가지게 되어 CPU를 덜 사용하게 된다. 이를 구현하기 위해 nice 변수를 thread 구조체 안에 선언하였으며, init_thread 함수에서 nice 값을 NICE_DEFAULT(=0)으로 초기화하도록 설정하였다.

```
struct thread
{
    ...
}
```

```

    int nice;
    ...
}

static void
init_thread (struct thread *t, const char *name, int priority)
{
    ...
    t->nice = NICE_DEFAULT;
    ...
}

```

nice 값을 설정하거나 가져올 때는 각각 `thread_set_nice`와 `thread_get_nice` 함수를 사용한다. 이러한 과정은 atomic하게 이루어져야 하므로, `intr_disable`을 통해 interrupt를 비활성화한 후, 모든 작업이 끝나면 다시 interrupt를 활성화해준다.

먼저, `thread_set_nice` 함수에서는 매개변수로 전달된 nice 값을 현재 thread의 nice 값으로 재설정한다. nice 값이 변경되었기 때문에, 이후에 설명할 priority 값도 변경된다. 이는 priority를 계산하는 수식에 nice 값이 포함되어 있기 때문이다. 따라서 priority를 다시 계산한 후, `swap_running_thread` 함수를 호출하여 현재 실행 중인 running thread와의 context switching이 필요한지 여부를 확인한다. 여기서 priority를 계산하는 함수(`mlfqs_calc_pri`)는 아래에서 설명할 예정이다.

```

void
thread_set_nice (int nice UNUSED)
{
    /* Not yet implemented. */
    enum intr_level old_level = intr_disable ();
    thread_current ()->nice = nice;
    mlfqs_calc_pri(thread_current());
    swap_running_thread ();
    intr_set_level (old_level);
}

```

그리고, `thread_get_nice` 는 현재 thread 의 nice 값을 반환한다.

```

int
thread_get_nice (void)
{
    /* Not yet implemented. */
    enum intr_level old_level = intr_disable ();
    int nice = thread_current ()-> nice;
    intr_set_level (old_level);
    return nice;
}

```

다음으로 priority 값은 0부터 63 사이의 값을 가지는 integer로, `recent_cpu`와 nice 값에 의해 결정된다. priority를 계산하는 수식은 아래와 같다.

```
priority = PRI_MAX - (recent_cpu / 4) - (nice * 2)
```

이 수식을 함수 형태로 구현하면 다음과 같다. 여기서 중요한 점은 recent_cpu와 load_avg가 실수(real value)로 계산되어야 하지만, Pintos는 floating point 연산을 지원하지 않기 때문에 이를 직접 구현해야 한다. 이와 관련된 세부 정보는 이후 recent_cpu와 load_avg 설명에서 추가할 예정이다. 이러한 추가적인 연산을 반영하여, 위의 수식을 "mlfqscalc_pri" 함수에 구현하였다.

```
void
mlfqscalc_pri(struct thread *t)
{
    if(t == idle_thread){return;}
    int calced_pri;
    int u,v,x;

    u = do_fp_calc((t->recent_cpu),-4,DIV_X_BY_N);
    v = PRI_MAX - 2 * (t->nice);
    x = do_fp_calc(u,v,ADD_X_AND_N);
    calced_pri = do_fp_calc(x,0,CONV_X_TO_INT_z);

    t->priority = calced_pri;
}
```

또한 priority는 4 ticks마다 재계산된다. 따라서 "mlfqscalc_pri2" 함수는 all_list에 있는 모든 thread 요소를 순회하면서, 각 thread의 priority를 위에서 설명한 "mlfqscalc_pri" 함수를 통해 재계산하도록 하였다. all_list는 시스템에 존재하는 모든 thread를 담고 있는 리스트이므로, 이 함수를 호출하면 시스템 내의 모든 thread의 priority가 최신 상태로 갱신된다. 이 함수는 나중에 설명할 timer_interrupt 함수에서 4 ticks마다 호출될 것이다.

```
void
mlfqscalc_pri2(void)
{
    struct list_elem *e;

    for (e = list_begin(&all_list); e != list_end(&all_list); e = list_next(e)) {
        struct thread *t = list_entry(e, struct thread, allelem);
        mlfqscalc_pri(t);
    }
}
```

앞서 설명한 것처럼, recent_cpu와 load_avg는 real value이기 때문에 이를 fixed-point 형식으로 변환한 후 연산을 진행해야 한다. Pintos는 기본적으로 floating point 연산을 지원하지 않기 때문에, 17.14 fixed-point 숫자 표현 방식을 사용하여 실수 연산을 구현할 것이다. 이 방식은 32비트 정수의 가장 오른쪽 14비트를 소수 부분으로 취급하는 것으로, 이를 통해 실수와 유사한 값을 정수 연산으로 처리할 수 있다.

17.14 형식은 17비트는 정수 부분, 14비트는 소수 부분을 나타내며, 부호 비트 1비트를 포함하여 총 32비트로 이루어져 있다. 덧셈과 뺄셈은 두 fixed-point 값의 경우 정수 덧셈과 뺄셈처럼 수행할 수 있지만, 곱셈과 나눗셈

은 조금 더 복잡하다. 두 fixed-point 값을 곱할 때는 결과값의 소수점이 14비트 왼쪽으로 치우치므로, 14비트를 오른쪽으로 시프트한 후 나눗셈을 수행해야 한다. 또한, 곱셈의 경우 64비트 연산을 사용하여 오버플로를 방지한다. 두 fixed-point 값을 나눌 때는 소수점이 14비트 오른쪽으로 치우치므로, 14비트를 왼쪽으로 시프트한 후 나누기를 수행한다. 이와 같이 나머지 연산들에 대해서도 pintos guide에서 제시하는 연산을 수행 할 수 있도록 아래와 같이 구현했다.

Convert n to fixed point:	$n * f$
Convert x to integer (rounding toward zero):	x / f
Convert x to integer (rounding to nearest):	$(x + f / 2) / f$ if $x \geq 0$, $(x - f / 2) / f$ if $x \leq 0$.
Add x and y :	$x + y$
Subtract y from x :	$x - y$
Add x and n :	$x + n * f$
Subtract n from x :	$x - n * f$
Multiply x by y :	$((\text{int64_t}) x) * y / f$
Multiply x by n :	$x * n$
Divide x by y :	$((\text{int64_t}) x) * f / y$
Divide x by n :	x / n

위의 사진에 나타난 식을 바탕으로 각 연산 모드를 case로 구분하고, 각 연산에 맞는 함수를 동적으로 호출하는 구조로 설계하였다. 이를 위해 binary_operation 함수 포인터를 사용하여, 연산 모드에 따라 적절한 연산 함수를 반환하도록 구현하였다.

```
int
calc_n_to_fp(int a, int b)
{
    (void)b;
    return F * a;
}

int
calc_x_to_int_z(int a, int b)
{
    (void)b;
    return a / F;
}

int
calc_x_to_int_n(int a, int b)
{

```

```
(void)b;
if(a >= 0){
    return (a + (F/ 2)) / F;
}else{
    return (a - (F / 2)) / F;
}
}

int
calc_add_x_y(int a, int b)
{
    return a + b;
}

int
calc_add_x_n(int a, int b)
{
    return a + b*F;
}

int
calc_sub_y_x(int a, int b)
{
    return a - b;
}

int
calc_sub_n_x(int a, int b)
{
    return a - b*F;
}

int
calc_mul_x_n(int a, int b)
{
    return a * b;
}

int
calc_mul_x_y(int a, int b)
{
    int result;
    result = (((int64_t) a) * b) / F;
    return result;
}

int
calc_div_x_n(int a, int b)
{
    return a / b;
}

int
calc_div_x_y(int a, int b)
```

```

{
    int result;
    result = (((int64_t) a) * F) / b;
    return result;
}

binary_operation get_operation(enum calc_mode mode) {
    switch (mode) {
        case CONV_N_TO_FP:
            return calc_n_to_fp;
        case CONV_X_TO_INT_Z:
            return calc_x_to_int_z;
        case CONV_X_TO_INT_N:
            return calc_x_to_int_n;
        case ADD_X_AND_Y:
            return calc_add_x_y;
        case ADD_X_AND_N:
            return calc_add_x_n;
        case SUB_Y_FROM_X:
            return calc_sub_y_x;
        case SUB_N_FROM_X:
            return calc_sub_n_x;
        case MUL_X_BY_N:
            return calc_mul_x_n;
        case MUL_X_BY_Y:
            return calc_mul_x_y;
        case DIV_X_BY_N:
            return calc_div_x_n;
        case DIV_X_BY_Y:
            return calc_div_x_y;
        default:
            return NULL;
    }
}

int
do_fp_calc(int x, int a, enum calc_mode mode)
{
    binary_operation operation = get_operation(mode);
    if(operation != NULL){
        return operation(x,a);
    }else{
        return 0;
    }
}

```

recent_cpu는 현재 thread가 CPU를 얼마나 점유했는지를 나타내는 real 값이다. 이 값이 크면 thread가 CPU를 오랜 시간 동안 점유하고 있다는 의미이다. 따라서 매 tick마다 이 값은 1씩 증가하며, 모든 thread의 recent_cpu 값은 매초마다 업데이트해주었다.

먼저, nice와 유사하게 recent_cpu 변수를 thread 구조체에 선언하고, init_thread 함수에서 이를 RECENT_CPU_DEFAULT (= 0) 으로 초기화하였다.

```

struct thread
{
    ...
    int recent_cpu;
    ...
}

static void
init_thread (struct thread *t, const char *name, int priority)
{
    ...
    t->recent_cpu = RECENT_CPU_DEFAULT;
    ...
}

```

recent_cpu 값을 가져오는 함수인 thread_get_recent_cpu는 현재 thread의 recent_cpu 값을 반환한다. 이 과정은 atomic하게 이루어져야 하기 때문에, 먼저 interrupt를 비활성화한 상태에서 recent_cpu 값을 가져온다. 이후 Pintos 문서의 지침에 따라 recent_cpu 값에 100을 곱한 후 정수형으로 변환하여 반환한다. 이 작업이 완료되면 interrupt를 다시 활성화한다.

```

int
thread_get_recent_cpu (void)
{
    /* Not yet implemented. */
    enum intr_level old_level = intr_disable ();
    int recent_cpu = do_fp_calc(do_fp_calc(thread_current ()-
>recent_cpu,100,MUL_X_BY_N),0,CONV_X_TO_INT_n);
    intr_set_level (old_level);
    return recent_cpu;
}

```

또한, recent_cpu를 계산하는 함수는 아래의 수식을 기반으로 mlfqs_calc_cpu 함수에 구현되어있다. 이 과정에서는 앞서 설명한 fixed-point 연산을 사용하여 실수 값을 계산하였다.

$$\text{recent_cpu} = (2 * \text{load_avg}) / (2 * \text{load_avg} + 1) * \text{recent_cpu} + \text{nice}$$

```

void mlfqs_calc_cpu(struct thread *t)
{
    if(t == idle_thread){return;}
    int calced_cpu;
    int u,v,w,x,y;

    u = do_fp_calc(load_avg ,2,MUL_X_BY_N);
    y = do_fp_calc(load_avg,2,MUL_X_BY_N);
    v = do_fp_calc(y,1,ADD_X_AND_N);
}

```



```

w = do_fp_calc(u,v,DIV_X_BY_Y);
x = do_fp_calc(w,t->recent_cpu,MUL_X_BY_Y);
calced_cpu = do_fp_calc(x,t->nice ,ADD_X_AND_N);

t->recent_cpu = calced_cpu;
}

```

앞서 설명했듯이, recent_cpu는 thread가 얼마나 오랫동안 CPU를 점유했는지를 나타내는 값이므로, 이 값은 mlfqs_incr_cpu 함수에서 매 tick마다 1씩 증가시켜주어야 한다. 이 함수는 timer_interrupt 에서 호출되고 이는 밑에서 설명할 예정이다.

```

void
mlfqs_incr_cpu(void)
{
    if(thread_current() != idle_thread){
        thread_current()-> recent_cpu = do_fp_calc(thread_current()-
>recent_cpu,1,ADD_X_AND_N);
    }
}

```

priority와 마찬가지로, mlfqs_calc_cpu2 함수에서 모든 thread를 순회하면서 각 thread의 recent_cpu 값을 새로운 값으로 업데이트해주었다. 이 과정에서는 all_list에 저장된 모든 thread를 순회하며, 각각의 recent_cpu 값을 mlfqs_calc_cpu 함수를 이용해 다시 계산한다.

```

void
mlfqs_calc_cpu2(void)
{
    struct list_elem *e;

    for (e = list_begin (&all_list); e != list_end (&all_list); e = list_next (e)) {
        struct thread *t = list_entry (e, struct thread, allelem);
        mlfqs_calc_cpu (t);
    }
}

```

마지막으로, load_avg는 1분 동안 수행 가능한 thread의 평균 개수를 의미한다. 이 값은 매초마다 업데이트되며, 시스템 전체의 CPU load를 나타낸다. 앞서 설명한 recent_cpu나 priority 값과 달리, load_avg는 thread별로 저장되는 값이 아니라 시스템 전역에서 관리되는 값이기 때문에, thread 구조체 내에 선언하지 않고 전역 변수로 정의하였다. 그리고 thread_start 함수 내에서 load_avg 값을 LOAD_AVG_DEFAULT (= 0)으로 초기화하였다.

```

int load_avg;

void
thread_start (void)
{

```

```

...
load_avg = LOAD_AVG_DEFAULT;
...
}

```

load_avg 값을 가져오는 함수인 thread_get_load_avg는 현재 시스템의 load_avg 값을 반환한다. 이 과정은 atomic하게 이루어져야 하므로, 먼저 interrupt를 비활성화한 상태에서 load_avg 값을 가져온다. 그런 다음, Pintos 문서의 지침에 따라 load_avg 값에 100을 곱한 후 정수형으로 변환하여 반환한다. 작업이 완료되면 interrupt를 다시 활성화한다.

```

int
thread_get_load_avg (void)
{
    /* Not yet implemented. */
    enum intr_level old_level = intr_disable ();
    int load_avg_value =
do_fp_calc(do_fp_calc(load_avg,100,MUL_X_BY_N),0,CONV_X_TO_INT_n);

    intr_set_level (old_level);
    return load_avg_value;
}

```

load_avg를 계산하는 함수는 아래의 수식을 기반으로 구현되었으며, fixed-point 연산을 사용하여 정확한 실수 값을 계산하였다. 수식에서 ready_threads는 현재 시점에서 실행 가능한 thread의 수를 의미한다. 여기서 중요한 점은 idle thread는 ready_threads 계산에 포함되지 않는다는 것이다. 따라서 만약 현재 thread가 idle thread 일 경우, ready_list 값을 그대로 사용하고, 그렇지 않은 경우 ready_list에 1을 더하여 현재 실행 중인 thread도 ready_threads에 포함될 수 있도록 하였다.

$$\text{load_avg} = (59/60) * \text{load_avg} + (1/60) * \text{ready_threads}$$

```

void
mlfqs_calc_ld(void)
{
    int num_t;
    int ld;
    if(thread_current() == idle_thread){
        num_t = list_size(&ready_list);
    }else{
        num_t = 1 + list_size(&ready_list);
    }

    int u, v, w, x, y;

    u = do_fp_calc(59,0,CONV_N_TO_FP);
    v = do_fp_calc(60,0,CONV_N_TO_FP);
    w = do_fp_calc(u,v,DIV_X_BY_Y);

```

```

    x = do_fp_calc(w,load_avg,MUL_X_BY_Y);

    u = do_fp_calc(1,0,CONV_N_TO_FP);
    //v = do_fp_calc(60,0,CONV_N_TO_FP);
    w = do_fp_calc(u,v,DIV_X_BY_Y);
    y = do_fp_calc(w,num_t,MUL_X_BY_N);

    ld = do_fp_calc(x,y,ADD_X_AND_Y);

    load_avg = ld;
}

```

이제 네 가지 변수인 nice, priority, recent_cpu, 그리고 load_avg에 대한 메서드 구현이 완료되었으며, 이를 기존 코드에 추가하였다. 앞서 설명한 것처럼, priority 값은 4 ticks마다 재계산되고, recent_cpu와 load_avg는 매초마다 업데이트된다. 그리고 매 tick마다 recent_cpu 값을 1씩 증가시켜주어야한다. 이를 반영하기 위해 tick 수를 증가시켜주는 아래의 timer_interrupt 함수에 조건문을 추가하여 위의 동작들을 구현하였다.

먼저, thread_mlfqs 변수를 통해 mlfqs 스케줄러가 활성화되었는지 확인한 후, 활성화된 경우에만 recent_cpu 값을 1씩 증가시키는 함수 mlfqs_incr_cpu를 호출한다. 그다음, 매 4 tick마다 priority 값을 재계산하기 위해 ticks % 4 == 0 조건을 추가하였다. 이 조건이 충족되면, mlfqs_calc_pri2 함수를 호출하여 모든 thread의 priority 값을 다시 계산한다.

또한, ticks % TIMER_FREQ == 0 조건을 통해 시스템의 recent_cpu와 load_avg 값을 매초마다 재계산하도록 설정하였다. TIMER_FREQ는 일반적으로 1초 동안 발생하는 tick 수를 의미하므로, 이 조건이 충족되면 mlfqs_calc_ld와 mlfqs_calc_cpu2() 함수를 호출하여 recent_cpu 와 load_avg 값이 재계산되도록 하였.

```

static void
timer_interrupt (struct intr_frame *args UNUSED)
{
    ticks++;
    thread_tick ();

    /*****/
    if (thread_mlfqs) {
        mlfqs_incr_cpu();
        if (ticks % 4 == 0) {
            mlfqs_calc_pri2();
            if (ticks % TIMER_FREQ == 0) {
                mlfqs_calc_ld();
                mlfqs_calc_cpu2();
            }
        }
    }
    /*****/

    thread_awake(ticks);
}

```

위의 timer_interrupt 함수까지 수정하고 나면 MLFQS 스케줄러 구현은 완료된다. 하지만, 두 번째 과제에서 구현했던 priority donation 부분이 MLFQS와 충돌하는 문제가 발생한다. priority donation은 MLFQS 스케줄러와 동시에 동작할 수 없기 때문에, MLFQS가 활성화된 경우에는 이 과정을 비활성화해주어야 한다.

이를 해결하기 위해, timer_interrupt 함수에서 사용했던 것처럼 thread_mlfqs라는 boolean 변수를 활용하여 MLFQS 스케줄러가 활성화되어 있는지를 확인하였다. thread_mlfqs가 활성화된 경우, priority donation 과정이 작동하지 않도록 if문을 이용해 비활성화 처리하였다.

```
void
lock_acquire (struct lock *lock)
{
    ...
    if(lock->holder != NULL){
        t->manager.target_lock = lock;
        list_insert_ordered(&lock->holder->manager.inheritor_list, &t-
>manager.inheritor, cmp_inheritor_pri, NULL);

        /*****/
        if(!thread_mlfqs){
            inherit_pri();
        }
        /*****/
    }
    ...
}

void
lock_release (struct lock *lock)
{
    ...
    /*****/
    if (!thread_mlfqs){
        release_helper(lock);
        restore_pri();
    }
    /*****/
    sema_up (&lock->semaphore);
}

void
thread_set_priority (int new_priority)
{
    /*****/
    if(thread_mlfqs){return;}
    /*****/

    thread_current ()->priority = new_priority;
    thread_current ()->manager.original_pri = new_priority;
    restore_pri();
    swap_running_thread();
}
```

Difference from design report

기존의 design report에서는 Pintos document의 Appendix B에 나와 있는 함수들을 그대로 작성하면 쉽게 구현이 가능할 것이라고 생각했다. 그러나 실제로 구현해보니, 매번 값이 업데이트될 때마다 모든 thread의 값이 정확히 업데이트되었음을 보장하기 위해 새로운 함수를 작성해야 했으며, floating point 연산을 처리하기 위해 따로 함수들을 구현해야 했다. 이로 인해 코드 자체의 복잡성(complexity)이 증가하였다.

또한, recent_cpu와 load_avg는 매 초마다 업데이트해야 하지만, timer_interrupt 함수에서는 ticks로 시간이 표현되어 있어 이를 초 단위로 변환하는 데에 어려움이 있었다. 이러한 부분에서 예상치 못한 구현상의 어려움을 겪었고, 추가적인 로직을 작성하여 해결해야 했다.

Limitations

MLFQS(Multi-Level Feedback Queue Scheduler)를 구현하는 과정은 Pintos 문서에 자세히 설명되어 있어, 이를 차근차근 따라가면 어렵지 않게 구현할 수 있었다. 하지만 floating point arithmetic을 다루는 부분에서 어려움이 있었다. recent_cpu와 load_avg 값을 계산할 때 고정소수점(fixed-point) 방식으로 변환하여 계산해야 했는데, 이 과정에서 코드가 복잡해지고 직관적이지 않게 작성되어, 디버깅 과정에서 어려움을 겪었다.

또한, MLFQS 스케줄러는 timer_interrupt에서만 우선순위를 처리해주는 것이 아니라, Priority Donation을 처리하는 부분에서도 예외 처리가 필요했다. 처음에는 Priority Donation과 MLFQS 간의 충돌을 이해하지 못해 어려움이 있었다. 하지만 구현을 진행하면서, MLFQS에서는 recent_cpu와 nice 값을 바탕으로 우선순위가 지속적으로 재계산되기 때문에, Priority Donation으로 인해 임시로 우선순위가 변경되면 충돌이 생긴다는 것을 알게 되었다. 따라서, Priority Donation 로직에서 MLFQS가 활성화된 경우 이 부분을 예외 처리를 한 결과, 계산 오류를 방지할 수 있었다.

Discussion

이번 세번째 과제를 통해 MLFQS 스케줄러를 구현하면서 priority, recent_cpu, nice, 그리고 load_avg 값들이 무엇인지에 대해서 알게 되었고, 이들이 서로 어떻게 상호작용하여 스케줄링에 영향을 미치는지 알게 되었다.

그리고 Pintos는 floating point 연산을 지원하지 않기 때문에 fixed-point 연산을 사용해야만 했는데, 이를 통해 실수 연산을 정수로 처리하는 방식과, recent_cpu 및 load_avg와 같은 값들을 정확하게 계산하기 위해 fixed-point 연산을 어떻게 적용해야 하는지를 배우게 되었다.

Result

```
pass tests/threads/mlfqs-block
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-chain
pass tests/threads/mlfqs-load-1
pass tests/threads/mlfqs-load-60
pass tests/threads/mlfqs-load-avg
pass tests/threads/mlfqs-recent-1
pass tests/threads/mlfqs-fair-2
pass tests/threads/mlfqs-fair-20
pass tests/threads/mlfqs-nice-2
pass tests/threads/mlfqs-nice-10
pass tests/threads/mlfqs-block
All 27 tests passed.
make[1]: Leaving directory '/root/pintos/src/threads/build'
root@a7173fe9bcc8:~/pintos/src/threads#
```

한